Data Mining Final Project Sprint 2 Documentation and Summary

Graduate Project Lead : Kiefer Armand
Undergraduate Modelers: Keahi Temple, William Becker, Reese Haydel
2/15/2026

Summary:

This week, our group focused on preparing for modeling as well as conducting the initial modeling work. The feature engineering described in the project outline was completed to simulate Command and Control (C2) scenarios. Prior to handing the dataset off to the modeling team, data cleaning, feature reduction, and normalization were applied to ensure consistency and model readiness.

The modeling effort centers on identifying situations that deviate from normal operating conditions. To support this goal, DBSCAN and Isolation Forest models were selected for comparison. The DBSCAN implementation, and the Isolation forest Implementation have been completed. Comparative analysis between the two approaches will be conducted early next week.

The following week will focus on security considerations, validation, mapping, and preparation for the final project phase. Documentation for each section completed this week, and article summaries are provided below.

Article Summaries - Kiefer armand

Reference 1 (For understanding Command and Control C2 systems)

"Understanding Command and Control," describes how most of the traditional C2 decision loops, like the observe, orient, decide, act cycle, developed under the assumptions of the Industrial Age, with communications viewed as a sequential process of centralized planning and decision-making. Most of these decision loops are based on doctrinal models of centralized planning, hierarchical dissemination of decisions, and common operational pictures, which influence decisionmaking in accordance with pre-defined procedures. (Alberts and Hayes, 2006, Chapter 8).

In contrast, the authors argue that C2 in the Information Age is defined by an abundance of information and dense, networked interactions. These conditions increasingly blur the boundaries between sensing, sensemaking, and acting, placing greater importance on continuous adaptation to emerging situations rather than strict adherence to predefined decision loops. Traditional models assume that planners can anticipate most relevant contingencies in advance and structure their decision processes accordingly, an assumption that limits agility in complex, uncertain, and rapidly changing environments (Alberts and Hayes, 2006, Chapter 8).

Reference 2 (For evaluation of C2 systems)

“Design of a Systems Theory Approach for the Evaluation of C2-Systems” , proposes a systems-theory-based framework for evaluating command and control (C2) systems in crisis and incident management contexts. The authors argue that large-scale incidents often expose limitations in existing C2 systems and that the integration of new technologies heightens the need for a comprehensive evaluation strategy capable of comparing different C2 systems or configurations.

To address this need, the paper adopts a systems theory perspective to integrate evaluation approaches drawn from multiple research domains into a single, cohesive framework that can be applied consistently across contexts. The proposed evaluation model is structured around three interconnected levels: physical properties, encompassing tools and technologies; structures and processes, including organizational hierarchies and decision-making workflows; and C2 system effectiveness, which reflects how well the system supports mission accomplishment. (Landsberg, Gleibs and Mudimu, 2023)

Reference 3 (Real World Example of dynamic improvement to Command and Control C2 systems)

This paper examines how eye-tracker-supported interactive decision support can enhance command and control (C2) performance within a simulated unmanned aerial vehicle (UAV) command-and-control environment. It frames UAV operation as a C2 problem that demands continuous sensemaking, prioritization, and coordination under significant time pressure. To address these demands, the proposed system analyzes the operator's visual attention in real time to identify moments when mission-critical information is being overlooked and provides subtle visual cues to redirect attention as needed. (Jei, Demir and Sasangohar, 2026)

The study's findings show that this adaptive decision support system improves overall mission performance by increasing alignment between the operator's attention and command intent—an essential requirement for effective C2. In contrast, conventional C2 interfaces rely on operators to self-manage attention across multiple displays, making them more vulnerable to breakdowns in situational awareness and delayed decision-making. By leveraging eye-tracking technology, the system enables a more dynamic form of C2 in which information flow is actively adjusted based on the commander's cognitive state. (Jei, Demir and Sasangohar, 2026)

Sources

Alberts, D.S. and Hayes, R.E. (2006) *Understanding command and control*. Washington, DC: Command and Control Research Program (CCRP), Department of Defense.

Landsberg, L., Gleibs, T. and Mudimu, O.A. (2023) 'Design of a systems theory approach for the evaluation of C2-systems', in Radianti, J., Dokas, I., LaLone, N. and Khazanchi, D. (eds.) *Proceedings of the 20th International ISCRAM Conference* (Command and Control Studies), University of Nebraska at Omaha, Omaha, pp. 692–700.

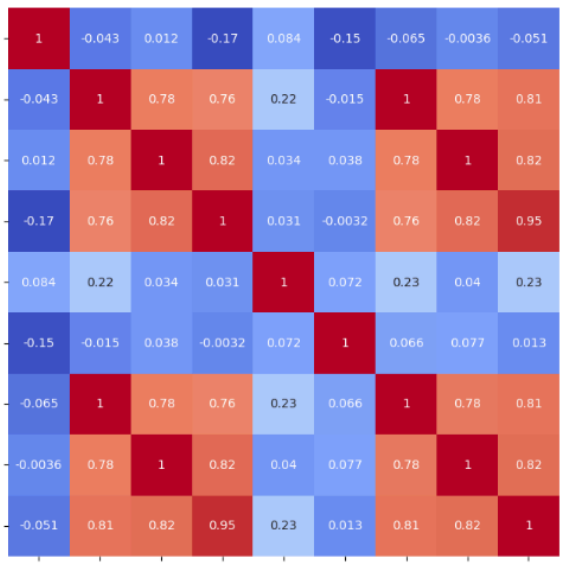
Jei, H.-G., Demir, M. and Sasangohar, F. (2026) 'Eyes on the mission: Mixed methods assessment of eye-tracker-enabled interactive decision support in a simulated unmanned aerial vehicle system', *arXiv*, 2601.18015 [Preprint].

Feature Engineering/Reduction/Normalization - William Becker & Kiefer Armand

Ground Truth data was used to create a higher dimension feature set of situations (multiple Individual datapoints) combined to represent a scenario at a given point in time.

	situation_id	total_detections	mean_x	mean_y	var_x	var_y	mean_bbox_area	centroid_x	centroid_y	mean_spatial_dispersion	mean_confidence
0	1	4	220.50	135.50	7079.00	2497.000000	860.25	236.875	148.375	79.907040	1.0
1	2	4	220.50	136.50	7079.00	2497.000000	860.25	236.875	149.375	79.907040	1.0
2	3	4	220.50	137.50	7079.00	2497.000000	860.25	236.875	150.375	79.907040	1.0
3	4	4	220.50	138.25	7079.00	2462.916667	860.25	236.875	151.125	79.666557	1.0
4	5	4	220.25	139.25	7148.25	2462.916667	860.25	236.625	152.125	79.918869	1.0

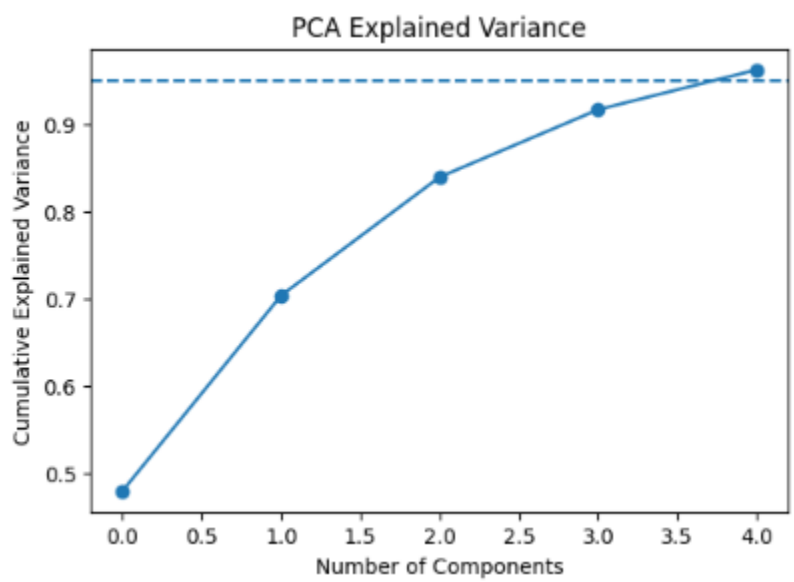
Feature Reduction was applied to the newly formed dataset via Feature correlation and feature value. Situation ID (unique identifier was removed as it contains the maximum variance for the dataset and only serves as an identifier. Confidence was removed as it contained 0 variance for the dataset. Centroid X and centroid y were removed as a result of high feature correlation:



Standard Scalar was chosen for this dataset as our model is unsupervised as intended to discover anomalous situations. Min Max has the potential to squash some of these outliers:

```
[[-0.68522467 -1.06299622 -0.7586931  -1.18702566 -0.65490643  0.26718234
  -1.16705713]
 [-0.68522467 -1.06299622 -0.7529173  -1.18702566 -0.65490643  0.26718234
  -1.16705713]
 [-0.68522467 -1.06299622 -0.7471415  -1.18702566 -0.65490643  0.26718234
  -1.16705713]
 [-0.68522467 -1.06299622 -0.74280964 -1.18702566 -0.66643483  0.26718234
  -1.16921753]
 [-0.68522467 -1.06396622 -0.73703384 -1.18008221 -0.66643483  0.26718234
  -1.16695087]
 [-0.68522467 -1.06396622 -0.73414594 -1.18008221 -0.6703036   0.26718234
  -1.16711639]
 [-0.68522467 -1.06299622 -0.72981409 -1.17325511 -0.68198915  0.26718234
  -1.16706739]
 [-0.68522467 -1.06299622 -0.72403829 -1.17325511 -0.68198915  0.28353361
  -1.16814096]
 [-0.68522467 -1.06396622 -0.72403829 -1.17346103 -0.68198915  0.27438701
  -1.16825431]
 [-0.68522467 -1.06396622 -0.72259433 -1.17346103 -0.68016965  0.2934425
  -1.16814386]]
```

PCA was applied to the dataset in order to explain 95 percent of all variance within the dataset while reducing the complexity of the model:



Modeling (Unsupervised -DBSCAN) - Keahi Temple

For this analysis, I have included comments provided by Colab Gemini while running the code.

1. Load Dataset

```python
import pandas as pd

# Load the dataset
df = pd.read_csv('/content/data_pca.csv')

# Display basic information and the first few rows
print(df.info())
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6592 entries, 0 to 6591
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   PC1     6592 non-null   float64
 1   PC2     6592 non-null   float64
 2   PC3     6592 non-null   float64
 3   PC4     6592 non-null   float64
 4   PC5     6592 non-null   float64
dtypes: float64(5)
memory usage: 257.6 KB
None
```

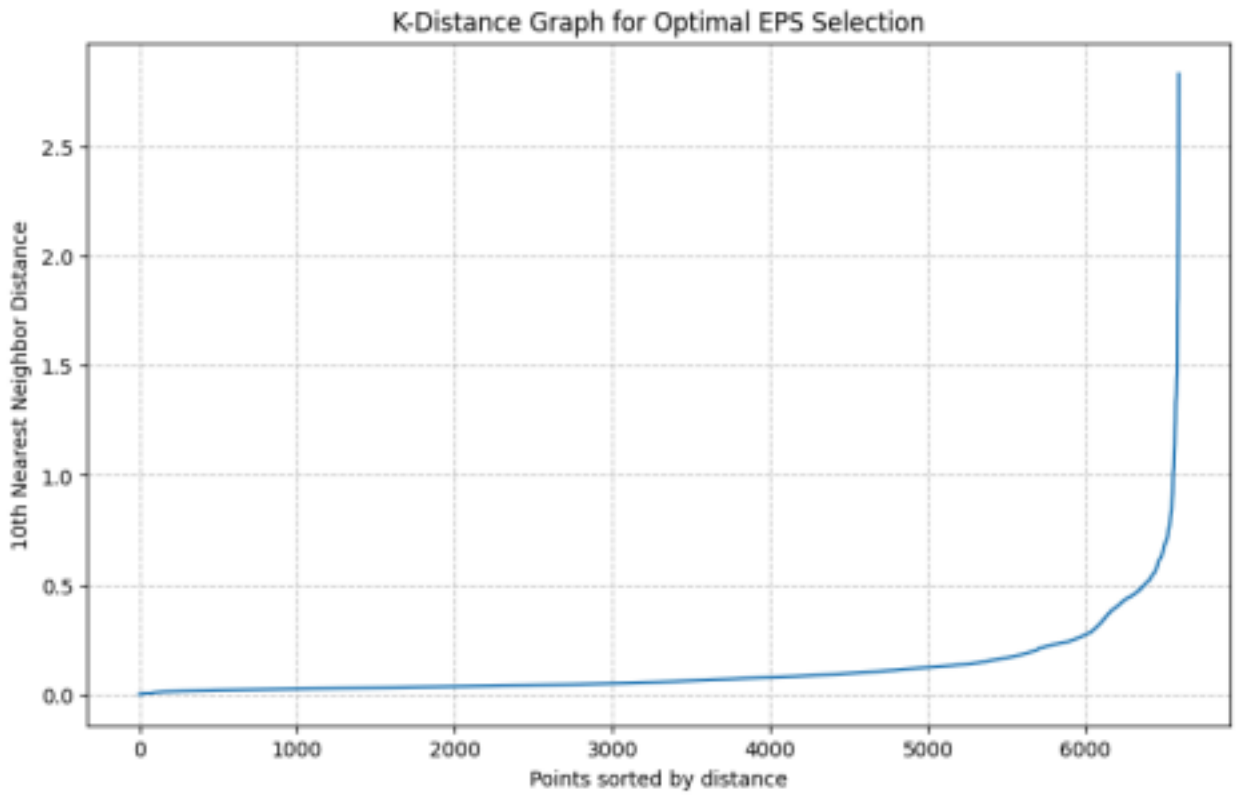
	PC1	PC2	PC3	PC4	PC5
0	-2.061702	-1.005418	-0.041884	-0.268034	0.307487
1	-2.058939	-1.004898	-0.042772	-0.269474	0.310782
2	-2.056176	-1.004378	-0.043661	-0.270914	0.314077
3	-2.056565	-1.008428	-0.054178	-0.270332	0.319285
4	-2.049814	-1.008308	-0.056238	-0.267982	0.318402

2. Perform DBSCAN/Fit Model & Generate Labels Parameter Selection

Subtask:

Determine the optimal 'eps' parameter for DBSCAN using a k-distance graph.

Reasoning: I will create a k-distance graph using NearestNeighbors to help determine the optimal 'eps' parameter for DBSCAN by identifying the 'elbow' point.



Apply DBSCAN/Attach Cluster Labels

Subtask:

Initialize and fit the DBSCAN clustering algorithm on the PCA dataset using an appropriate 'eps' value identified from the k-distance graph.

Optimal Parameters: (eps=0.35, min_samples=10)

Reasoning: I will initialize and fit the DBSCAN algorithm using an eps value of 0.35 (determined from the elbow in the k-distance graph) and min_samples=10, then add the resulting labels to the dataframe.

```python
from sklearn.cluster import DBSCAN

# Initialize DBSCAN with eps=0.35 and min_samples=10
# 0.35 is chosen as a representative elbow point from the k-distance graph
dbscan = DBSCAN(eps=0.35, min_samples=10)

# Fit the model and extract labels
df['cluster_label'] = dbscan.fit_predict(df)

# Print unique labels to check clusters and noise
unique_labels = df['cluster_label'].unique()
print(f"Unique cluster labels found: {sorted(unique_labels)}")
print(f"Number of clusters (excluding noise): {len([l for l in unique_labels if l != -1])}")
print(f"Noise detected: {-1 in unique_labels}")
```

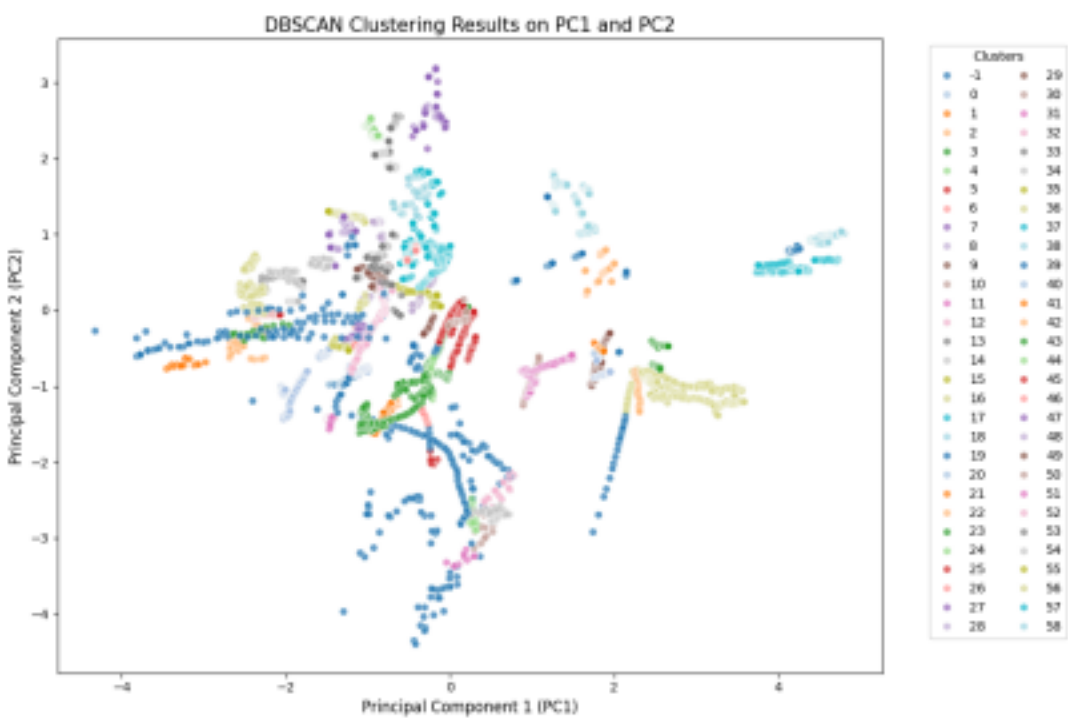
```
Unique cluster labels found: [np.int64(-1), np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7),
Number of clusters (excluding noise): 59
Noise detected: True
```

Visualize Clusters

Subtask:

Create a scatter plot of the first two principal components (PC1 and PC2) colored by the DBSCAN cluster labels.

Reasoning: I will create a scatter plot of PC1 and PC2 using seaborn, coloring the points by their DBSCAN cluster labels to visualize the clustering results.



Final Task

Subtask:

Summarize the clustering results, including the number of clusters identified and the proportion of data points classified as noise.

Summary:

Q&A

How many clusters were identified by the DBSCAN algorithm?
The algorithm identified 59 distinct clusters (labeled 0 through 58).

Was noise detected in the dataset, and how was it represented?
Yes, noise was detected and represented by the cluster label -1.

Data Analysis Key Findings

● **Optimal Parameter Selection**: Using a k-distance graph with a heuristic of 10 nearest neighbors (2 * dimensions), an elbow point was identified at an `eps` value of 0.35. ● **Clustering Complexity**: The DBSCAN algorithm categorized the PCA-transformed data into 59 separate clusters, indicating a high degree of local density variation within the dataset.

● **Spatial Distribution**: Visualization of the first two principal components (PC1 and PC2) confirmed that the density-based approach effectively separated concentrated data points from sparse noise.

● **Noise Presence**: The model successfully differentiated outliers from the primary clusters, assigning them a specific noise label to prevent them from distorting the cluster characteristics.

3. Quantify Detection Results

Summarize Cluster Metrics

Subtask:

Calculate the count of data points per cluster **(59)** and the overall noise percentage from the DBSCAN results.

Reasoning: I will calculate the distribution of points across the DBSCAN clusters and determine the exact percentage of noise in the dataset to provide a quantitative summary of the clustering performance.

```
Name: count, dtype: int64

============================
Total data points: 6592
Noise points: 345
Noise Percentage: 5.23%
```

```
""  Number of data points per cluster (Label -1 is noise):
    cluster_label
    -1     345
     0     315
     1      17
     2      51
     3      18
     4     279
     5     153
     6      12
     7      38
     8      13
     9      54
    10      15
    11      10
    12      19
    13     125
    14      16
    15      49
    16      12
    17     388
    18      95
    19     157
    20     216
    21     225
    22      50
    23     348
    24      48
    25      12
    26      15
    27     115
    28      34
    29     103
    30      53
    31     172
    32      86
    33     172
    34      96
    35     195
    36     427
    37     235
    38     282
    39      46
    40      62
    41      18
    42      33
    43      42
    44      48
    45     128
    46      38
    47      98
    48      30
    49      10
    50      26
    51      14
    52     141
    53     164
    54      53
```

4.Hyperparameter Tuning + ANALYSIS

Subtask:

Perform a systematic search over 'eps' and 'min_samples' values to identify a more stable

DBSCAN configuration for the PCA data.

Reasoning: I will perform a systematic search over a range of 'eps' and 'min_samples' values to evaluate their impact on the number of clusters and noise percentage, helping to find a more stable configuration.

```python
import pandas as pd
from sklearn.cluster import DBSCAN
import numpy as np

# Define the range for eps and min_samples
eps_values = [0.3, 0.4, 0.5, 0.6, 0.7]
min_samples_values = [5, 10, 20, 50]

# Features to use for clustering
features = df[['PC1', 'PC2', 'PC3', 'PC4', 'PC5']]

results = []

# Iterate through parameter combinations
for eps in eps_values:
    for min_samples in min_samples_values:
        dbscan = DBSCAN(eps=eps, min_samples=min_samples)
        labels = dbscan.fit_predict(features)

        # Calculate metrics
        unique_labels = set(labels)
        n_clusters = len([l for l in unique_labels if l != -1])
        n_noise = list(labels).count(-1)
        noise_perc = (n_noise / len(labels)) * 100

        results.append({
            'eps': eps,
            'min_samples': min_samples,
            'n_clusters': n_clusters,
            'noise_percentage': round(noise_perc, 2)
        })

# Display the results in a DataFrame for easy comparison
tuning_df = pd.DataFrame(results)
print("DBSCAN Hyperparameter Tuning Results:")
display(tuning_df)
```

DBSCAN Hyperparameter Tuning Results:

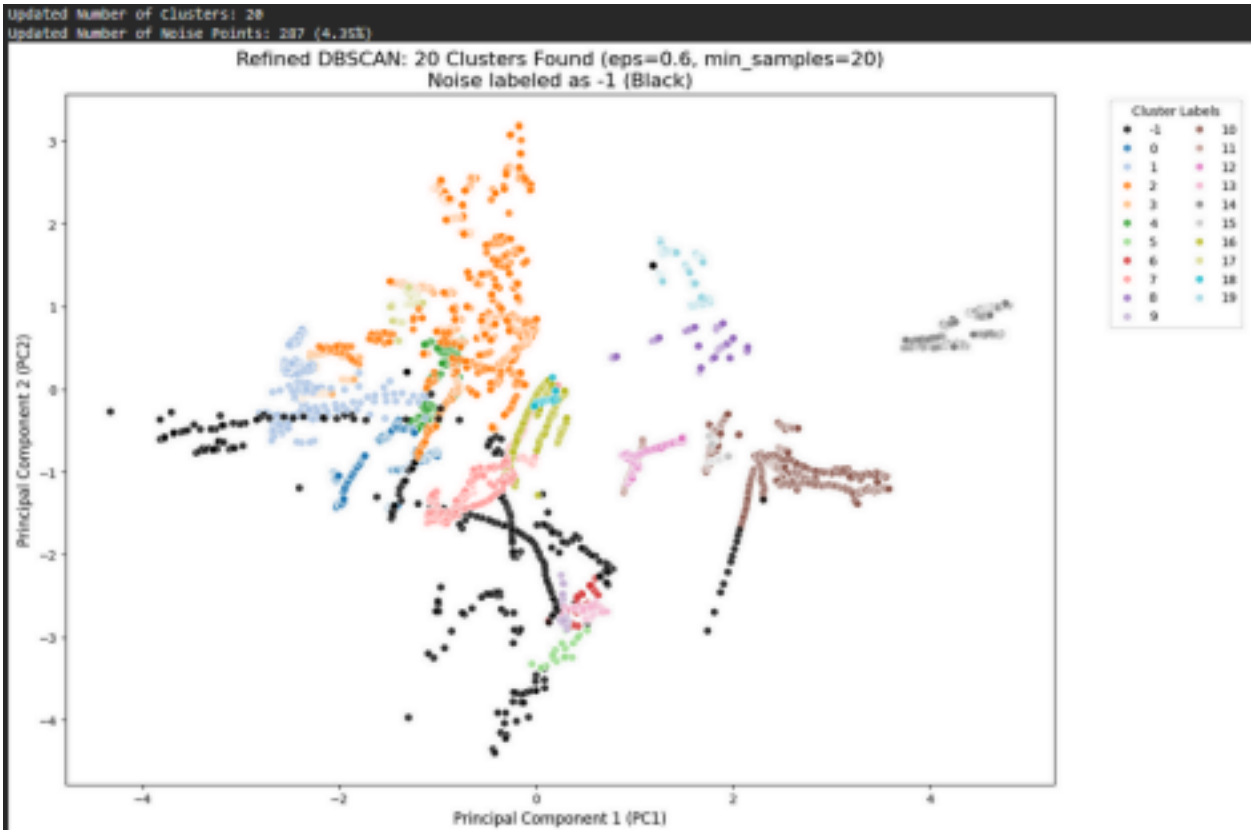
	eps	min_samples	n_clusters	noise_percentage
0	0.3	5	89	1.81
1	0.3	10	70	6.57
2	0.3	20	53	12.03
3	0.3	50	32	32.99
4	0.4	5	64	0.94
5	0.4	10	57	4.07
6	0.4	20	43	8.83
7	0.4	50	32	24.61
8	0.5	5	34	0.55
9	0.5	10	34	1.76
10	0.5	20	27	6.37
11	0.5	50	25	14.64
12	0.6	5	24	0.35
13	0.6	10	25	0.97
14	0.6	20	20	4.35
15	0.6	50	21	9.60
16	0.7	5	19	0.23
17	0.7	10	19	0.56
18	0.7	20	18	2.78
19	0.7	50	13	7.90

Analyze Optimal Results

Subtask:

Refit the DBSCAN model using the optimal parameters (eps=0.6, min_samples=20) and visualize the updated cluster distribution.

Reasoning: I will refit the DBSCAN model using the optimal parameters (eps=0.6, min_samples=20), calculate the updated cluster metrics, and visualize the results to verify the improved cluster structure.



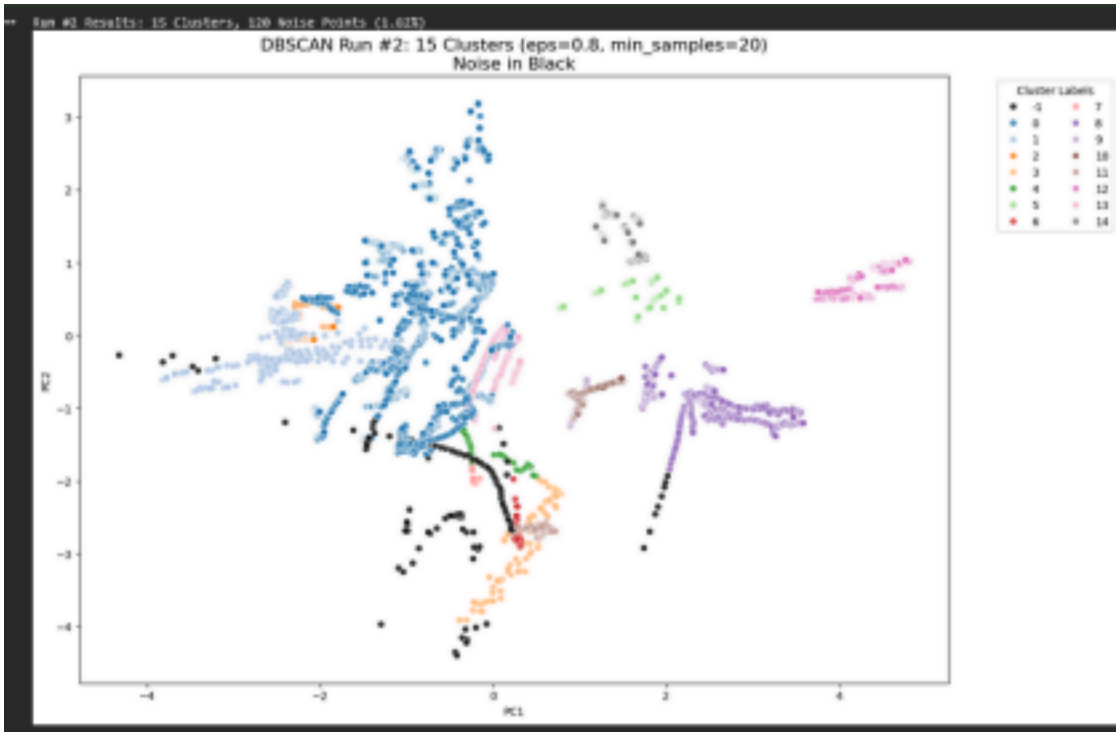
This initial run is quite solid, having distinct clustering and a low amount of noise. However, I need to fine tune it and see if I can get something better.

Repeating Tuning

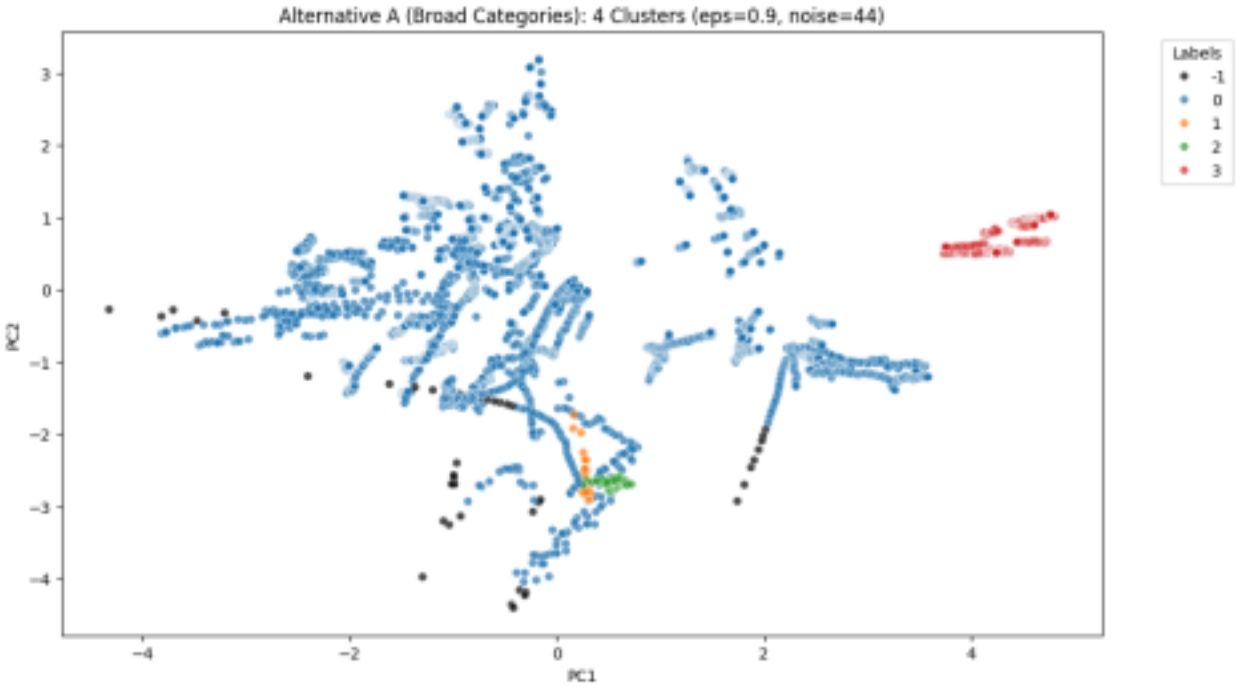
```python
import pandas as pd
from sklearn.cluster import DBSCAN
import numpy as np

# Expanded range for eps and min_samples to further reduce noise
eps_values = [0.8, 0.9, 1.0, 1.1, 1.2]
min_samples_values = [10, 20, 30, 50]

# Features to use for clustering
features = df[['PC1', 'PC2', 'PC3', 'PC4', 'PC5']]

results_new = []

# Iterate through parameter combinations
for eps in eps_values:
    for min_samples in min_samples_values:
        dbscan = DBSCAN(eps=eps, min_samples=min_samples)
        labels = dbscan.fit_predict(features)

        # Calculate metrics
        unique_labels = set(labels)
        n_clusters = len([l for l in unique_labels if l != -1])
        n_noise = list(labels).count(-1)
        noise_perc = (n_noise / len(labels)) * 100

        results_new.append({
            'eps': eps,
            'min_samples': min_samples,
            'n_clusters': n_clusters,
            'noise_percentage': round(noise_perc, 3)
        })

# Display the results
tuning_df_expanded = pd.DataFrame(results_new)
print("New Expanded DBSCAN Tuning Results (Run #2):")
display(tuning_df_expanded)
```

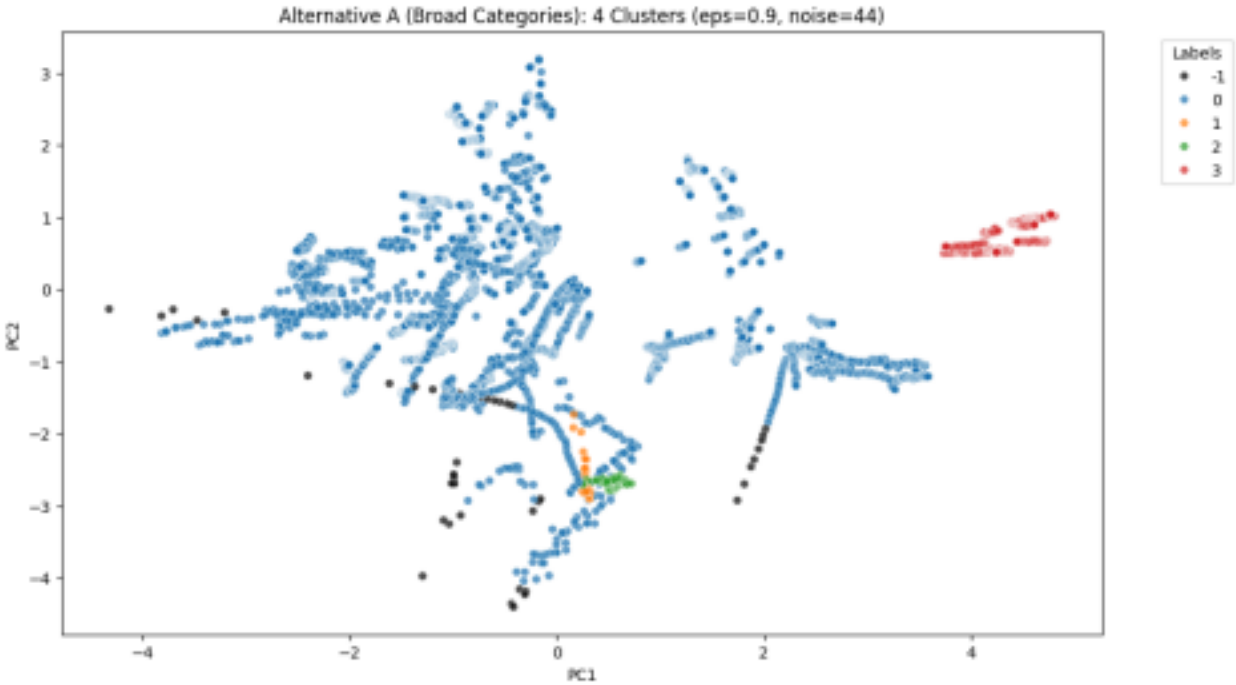
```
New Expanded DBSCAN Tuning Results (Run #2):
```

	eps	min_samples	n_clusters	noise_percentage
0	0.8	10	13	0.44
1	0.8	20	15	1.82
2	0.8	30	13	3.79
3	0.8	50	13	4.70
4	0.9	10	4	0.30
5	0.9	20	4	0.67
6	0.9	30	6	2.23
7	0.9	50	7	4.14
8	1.0	10	2	0.26
9	1.0	20	3	0.47
10	1.0	30	3	1.20
11	1.0	50	4	3.28
12	1.1	10	2	0.18
13	1.1	20	3	0.38
14	1.1	30	3	0.86
15	1.1	50	3	1.74
16	1.2	10	2	0.06
17	1.2	20	2	0.29
18	1.2	30	2	0.64
19	1.2	50	3	1.32

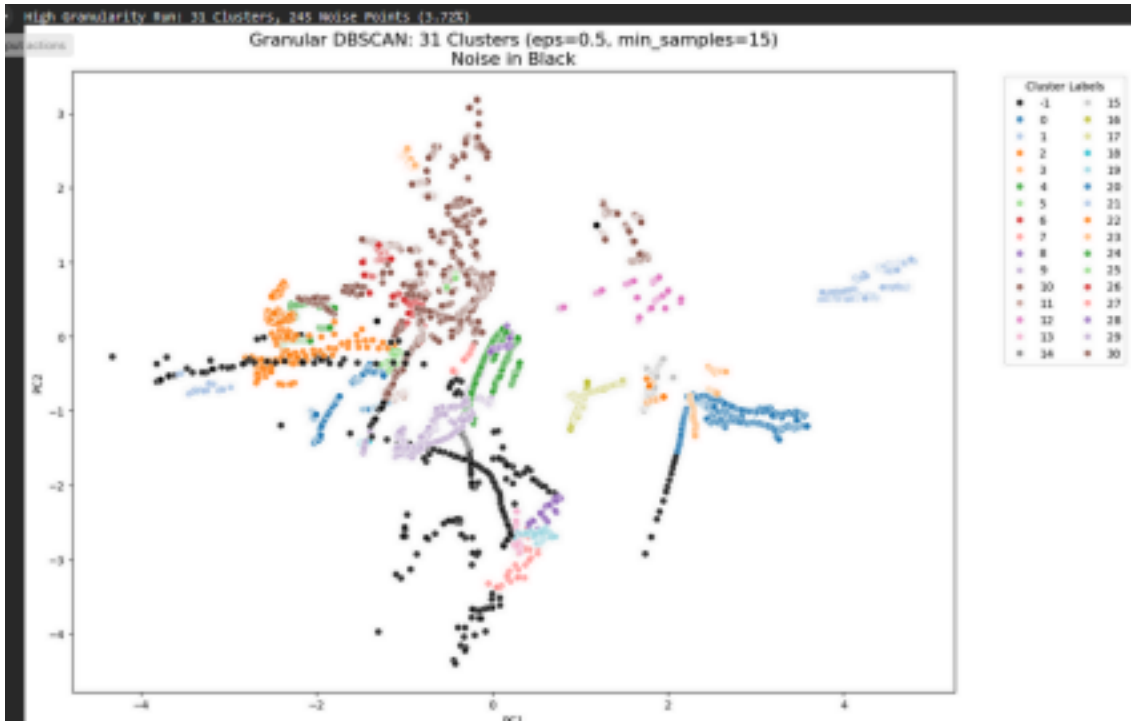


This could be useful. It ends up grouping quite a large amount of points into one singular island but has a very low amount of noise. By increasing epsilon, the previously dense pockets merged into that one island, and the neighboring islands have become much cleaner.





These last 2 are not very useful, the clustering is far too high and all distinct structures are lost. Very low amount of noise but the groupings are not valuable at all.



Higher amount of noise, but more distinct structuring. Not sure which between these 2 would be more useful but in my opinion it is between these 2 and the 15 clusters output. The 15 clusters output is easy to interpret and reduces the noise quite a lot, but these ones more accurately represent high density neighborhoods and preserve their local structure.

Modeling – Isolation Forest - Reese Haydel

Step 1: Load datasets

```
from google.colab import files
uploaded = files.upload()  # upload train.csv and test.csv

train_df = pd.read_csv("train_pca.csv")
test_df  = pd.read_csv("test_pca.csv")

print(train_df.shape, test_df.shape)
train_df.head()
```

```
Choose Files  2 files
test_pca.csv(text/csv) - 129647 bytes, last modified: 2/14/2026 - 100% done
train_pca.csv(text/csv) - 518679 bytes, last modified: 2/14/2026 - 100% done
Saving test_pca.csv to test_pca.csv
Saving train_pca.csv to train_pca.csv
(5273, 5) (1319, 5)
```

	PC1	PC2	PC3	PC4	PC5
0	4.367831	0.515289	-1.194801	-1.107948	0.375456
1	-0.200170	-4.017518	-4.261421	2.091311	0.208783
2	0.983616	-1.082756	-0.652174	0.137405	-0.654660
3	4.203627	0.792488	-0.829972	-0.500587	-0.131040
4	-0.412733	2.389653	-0.472388	1.166475	0.298874

Loaded the preprocessed PCA datasets into pandas DataFrames for training and testing. Verified successful import by printing dataset dimensions and previewing rows/columns to confirm the expected PCA feature set (PC1–PC5) and numeric formatting. This confirms the data is ready to be converted into a model input matrix.

Step 2: Select Algorithm (Isolation forest)

```
# --- Modeling ---
from sklearn.ensemble import IsolationForest
```

```
iso = IsolationForest(
    n_estimators=500,
    contamination=0.05,
    max_samples="auto",
    max_features=1.0,
    bootstrap=False,
    n_jobs=-1,
    random_state=42
)
```

Selected Isolation Forest as the unsupervised anomaly detection algorithm because it is effective at isolating rare observations in feature space without requiring labels. Configured the model with key hyperparameters (e.g., number of trees and contamination rate) to control stability and the expected fraction of anomalies.

Step 3: Fit Model (train)

```
X_train = train_df.values
X_test  = test_df.values

iso.fit(X_train)
```

```
IsolationForest
IsolationForest(contamination=0.05, n_estimators=500, n_jobs=-1,
                random_state=42)
```

The call iso.fit(X_train) builds an ensemble of 500 randomized isolation trees using only the training data to learn the baseline 'normal' structure of the PCA feature space. Because fit() returns the trained estimator, Google Colab automatically prints the model summary in the
output cell (showing key hyperparameters such as contamination=0.05, n_estimators=500, n_jobs=-1, and random_state=42), confirming the model was successfully fitted and is ready for prediction and scoring on train/test data.

Step 4: Evaluate Training Data (1 = normal) (-1 = anomaly)

```
train_preds = iso.predict(X_train)  # 1 normal, -1 anomaly
train_anomalies = (train_preds == -1).sum()
print("Training anomalies:", train_anomalies, "/", len(train_preds))
print("Training anomaly rate:", train_anomalies / len(train_preds))

Training anomalies: 264 / 5273
Training anomaly rate: 0.050066375877109805
```

Ran the trained model on the training set to obtain binary anomaly labels where 1 indicates normal and -1 indicates anomaly. Counted the number of training anomalies flagged to quantify how many points are considered outliers under the current threshold settings (primarily driven by the contamination parameter). As shown in the image above, the number of training anomalies was 264/5273 with an anomaly rate of ~0.0500066.

Step 5: Evaluate on Test Data

```
test_preds = iso.predict(X_test)
test_anomalies = (test_preds == -1).sum()
print("Test anomalies:", test_anomalies, "/", len(test_preds))
print("Test anomaly rate:", test_anomalies / len(test_preds))

Test anomalies: 77 / 1319
Test anomaly rate: 0.058377558756633814
```

Applied the trained Isolation Forest to the test set to generate anomaly labels and counted the flagged anomalies. Comparing the anomaly rate on test vs train provides a quick check for distribution shift or over/under-sensitivity of the chosen parameters. As shown in the image above, the number of training anomalies was 77/1319 with an anomaly rate of ~0.05837756.

Step 6: Get Anomaly Scores

```
# Continuous scores
train_score_samples = iso.score_samples(X_train)       # lower => more anomalous
train_decision      = iso.decision_function(X_train)   # higher => more normal, ~0 boundary

test_score_samples  = iso.score_samples(X_test)
test_decision       = iso.decision_function(X_test)

print("Train score_samples range:", train_score_samples.min(), train_score_samples.max())
print("Train decision_function range:", train_decision.min(), train_decision.max())


Train score_samples range: -0.7046131900195411 -0.3860843097154078
Train decision_function range: -0.14370089796801155 0.1748279823361218
```

During this step, we computed continuous anomaly scores so observations can be ranked by severity rather than relying only on the binary label output. We used iso.score_samples(X) to obtain raw Isolation Forest scores, where lower values indicate more anomalous points in sklearn. We also computed iso.decision_function(X), which is a thresholded scoring scale where the decision boundary is at 0: values < 0 are classified as anomalies and values > 0 are classified as normal under the chosen contamination setting. For the training set, the score ranges were score_samples ∈ [-0.7046 and -0.3861] and decision_function ∈ [-0.1437, 0.1748], confirming that the most anomalous points fall into the lower tail of score_samples and the negative region of decision_function.

Step 7: Parameter Tuning

	trial	contamination	n_estimators	max_features	max_samples	train_anom_rate	train_sep	test_anom_rate	test_sep	rank_score
34	34	0.01	1200	0.85	auto	0.010051	0.201516	0.012889	0.202443	0.196666
8	8	0.01	200	0.85	0.8	0.010051	0.196781	0.015163	0.196175	0.185849
12	12	0.01	800	0.50	0.8	0.010051	0.172177	0.015163	0.175449	0.165123
30	30	0.01	1200	0.50	1.0	0.010051	0.166864	0.014405	0.172900	0.164090
19	19	0.02	1200	1.00	0.6	0.020102	0.169634	0.025777	0.175386	0.163832
38	38	0.03	200	0.85	auto	0.030154	0.182171	0.040182	0.182948	0.162584
23	23	0.02	500	1.00	0.6	0.020102	0.168180	0.026535	0.172541	0.159470
22	22	0.02	500	1.00	0.6	0.020102	0.168180	0.026535	0.172541	0.159470
9	9	0.02	1200	1.00	1.0	0.020102	0.163828	0.025777	0.170513	0.158959
27	27	0.02	200	1.00	1.0	0.020102	0.163350	0.025777	0.169535	0.157980
17	17	0.05	800	1.00	auto	0.050066	0.167949	0.059136	0.167891	0.149619
28	28	0.05	200	0.50	auto	0.050066	0.163814	0.059136	0.166135	0.147863
15	15	0.02	500	0.50	0.6	0.020102	0.153079	0.025019	0.157821	0.147783
2	2	0.03	800	0.85	0.8	0.030154	0.152550	0.035633	0.158411	0.147144
11	11	0.03	1200	0.85	1.0	0.030154	0.146838	0.034875	0.154446	0.144696

During this step, multiple Isolation Forest trials were ran with varying contamination, n_estimators, max_features, and max_samples. For each trial, the anomaly rate on train/test (train_anom_rate, test_anom_rate) and a score-separation metric (train_sep, test_sep) that measures how distinctly the model separates predicted anomalies from predicted normals in continuous score space was recorded. Trials were ranked using rank_score, prioritizing strong test-set separation while avoiding unrealistic anomaly rates.

Based on this tuning table, the top configuration was **Trial 34** with: contamination = 0.01, n_estimators = 1200, max_features = 0.85, and max_samples = "auto". This setting achieved the highest rank_score (0.196666) and the strongest test_sep (0.202443), producing approximately 1.01% anomalies on train (train_anom_rate = 0.010051) and 1.29% on test (test_anom_rate = 0.012889). These hyperparameters provided the clearest separation on unseen data while maintaining an interpretable anomaly threshold consistent with a 1% contamination assumption.

Step 8: Identify/Inspect

```
# Most anomalous rows in TRAIN
train_anom_view = train_out.sort_values("if_score_samples", ascending=True).head(25)
train_anom_view
```

	PC1	PC2	PC3	PC4	PC5	if_pred	if_score_samples	if_decision
3687	-4.376196	-0.282873	1.934888	-4.262284	3.294898	-1	-0.784853	-0.163701
542	-0.423217	-4.484636	-5.176278	2.066299	0.661634	-1	-0.684172	-0.133268
2568	-2.436757	-1.264487	3.194647	-2.444383	2.935742	-1	-0.683827	-0.133015
1622	-0.311718	-4.228051	-4.677632	2.174412	0.344582	-1	-0.680583	-0.132678
3473	-0.361742	-4.157663	-4.733828	2.068292	0.459921	-1	-0.690875	-0.129863
2863	-0.334186	-4.180359	-4.661774	1.937621	0.518846	-1	-0.687237	-0.126325
1	-0.290178	-4.017578	-4.261421	2.091311	0.298783	-1	-0.686381	-0.125468
3973	-3.515104	-0.438353	1.463856	-3.261787	2.154617	-1	-0.684119	-0.123206
3784	-1.304748	-3.975546	-4.168111	1.671571	0.881916	-1	-0.683650	-0.123838
2130	-0.310747	-3.917474	-3.841308	2.011851	0.240879	-1	-0.678771	-0.117798
2486	-0.077908	-3.969783	-4.292203	1.916785	0.370282	-1	-0.678734	-0.117791
5089	-0.360503	-3.915769	-3.835317	1.960238	0.283382	-1	-0.677831	-0.116119
3598	-0.153218	-3.788624	-3.967853	1.902531	0.228339	-1	-0.671901	-0.110888
3580	-2.880118	-0.319047	1.606917	-2.943931	1.888537	-1	-0.681601	-0.100668
1536	-0.128738	-3.796739	-3.805408	1.689689	0.381942	-1	-0.681443	-0.100531
4525	-1.638127	-1.312247	2.900726	-1.589640	2.038857	-1	-0.657054	-0.096101
737	-0.228198	-3.668665	-3.306363	1.872027	0.143882	-1	-0.656631	-0.096019
1882	-0.197778	-3.677686	-3.295599	1.856583	0.167039	-1	-0.654380	-0.093358
1380	1.745199	-2.933489	-3.238823	0.705624	1.486833	-1	-0.651909	-0.090997
1574	-0.151327	-3.693613	-3.588633	1.683710	0.382661	-1	-0.650625	-0.090013
3121	-0.198313	-3.675471	-3.803429	1.696250	0.298872	-1	-0.650418	-0.089906
1180	-2.840605	-0.356119	1.369153	-2.712341	1.614780	-1	-0.649132	-0.088220
4874	-2.512831	-0.353984	1.609415	-2.732437	1.638633	-1	-0.648861	-0.085148
1282	-3.208638	-0.445439	1.807101	-2.685285	1.540348	-1	-0.648654	-0.085142
5223	-0.003844	-3.662235	-3.613804	1.582588	0.318170	-1	-0.648665	-0.083753

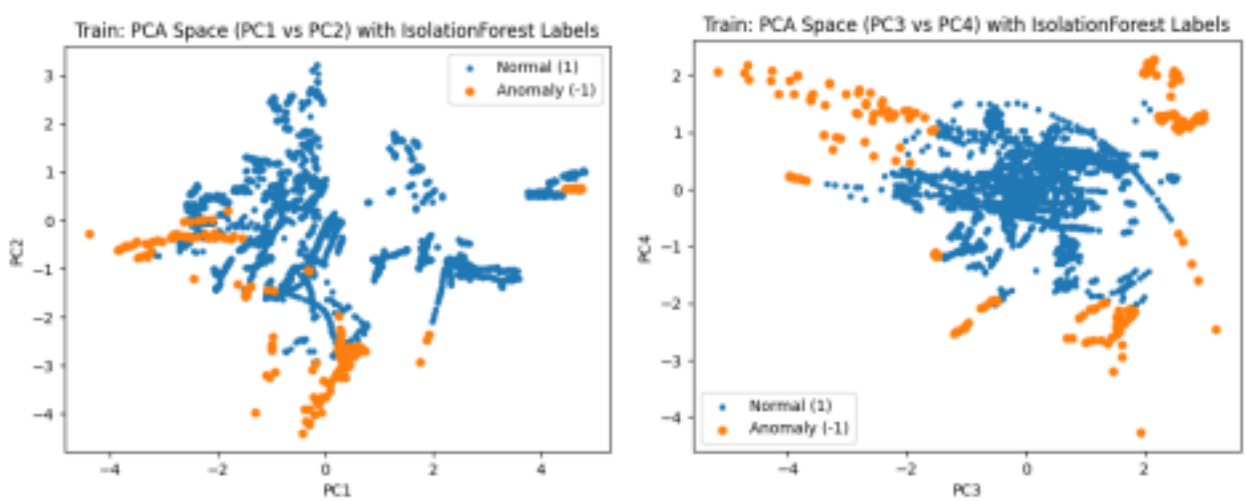
The most severe anomalies in the training dataset were inspected by ranking observations using Isolation Forest's continuous anomaly score. The code sorts train_out by if_score_samples in ascending order and displays the top 25 rows. Because sklearn's score_samples() assigns lower values to more anomalous points, the rows shown represent the most extreme outliers identified by the model in the training data. Many of the top-ranked anomalies show extreme PCA values, such as very negative PC4 (Ex. around -4.26) paired with high positive PC5 (Ex. around 3.29), or unusually large-magnitude values across multiple components. This suggests the model is flagging points located far from the dense 'normal' region of the PCA feature space.

```
# Most anomalous rows in TEST
test_anom_view = test_out.sort_values("if_score_samples", ascending=True).head(25)
test_anom_view
```

	PC1	PC2	PC3	PC4	PC5	if_pred	if_score_samples	if_decision
1272	-3.864968	-8.377755	1.987662	-3.843221	2.644493	-1	-0.695037	-0.138025
1806	-3.751832	-8.289379	1.768758	-3.720921	2.668136	-1	-0.698944	-0.138031
1888	-0.642658	-4.394620	-5.197203	1.995087	0.613673	-1	-0.692169	-0.131258
573	-3.248001	-8.330180	1.716525	-3.302214	2.246417	-1	-0.684092	-0.123988
1302	-0.322734	-4.043026	-4.197399	1.947085	0.411899	-1	-0.680449	-0.119537
1941	-0.234658	-3.781447	-3.538389	1.857948	0.257257	-1	-0.660859	-0.099947
111	-0.084658	-3.536051	-3.153778	1.864192	-0.017778	-1	-0.654019	-0.093107
843	0.088347	-3.618323	-3.398415	1.783340	0.202358	-1	-0.651919	-0.091007
1309	0.080385	-3.481586	-3.214467	1.748083	0.074252	-1	-0.649154	-0.088242
23	-2.987879	-8.417543	1.212081	-2.681173	1.565078	-1	-0.646788	-0.085875
523	-2.743463	-8.356294	1.408448	-2.672329	1.594296	-1	-0.643264	-0.082352
1289	0.624992	-2.608322	2.598241	1.839953	2.000009	-1	-0.641371	-0.080458
315	0.623505	-2.619734	2.629068	1.873539	1.996377	-1	-0.640681	-0.079779
279	0.682332	-2.686158	2.623551	1.957274	1.996268	-1	-0.640464	-0.079562
715	0.628659	-2.701436	2.566664	1.188271	1.972567	-1	-0.639184	-0.078271
369	0.683333	-2.685434	2.602341	1.895634	2.001851	-1	-0.638862	-0.077758
798	0.614809	-2.578805	2.658548	1.858506	1.915148	-1	-0.638487	-0.077584
651	-3.224212	-8.447219	0.891644	-2.637361	1.442723	-1	-0.638313	-0.077401
1265	0.324813	-2.866687	2.153520	2.278664	1.131561	-1	-0.637467	-0.076555
1138	-3.448009	-8.492881	0.629689	-2.659606	1.436888	-1	-0.636192	-0.075279
397	0.312588	-2.874070	2.137541	2.263040	1.112548	-1	-0.633781	-0.072868
644	0.558029	-2.785779	2.508015	1.268974	1.848521	-1	-0.632911	-0.071998
818	0.511912	-2.847763	2.817483	1.152268	1.832567	-1	-0.632665	-0.071743
94	0.321478	-2.856971	2.121041	2.216089	1.127461	-1	-0.631289	-0.070378
1230	0.380616	-2.650527	2.738422	1.135482	1.827467	-1	-0.629873	-0.068968

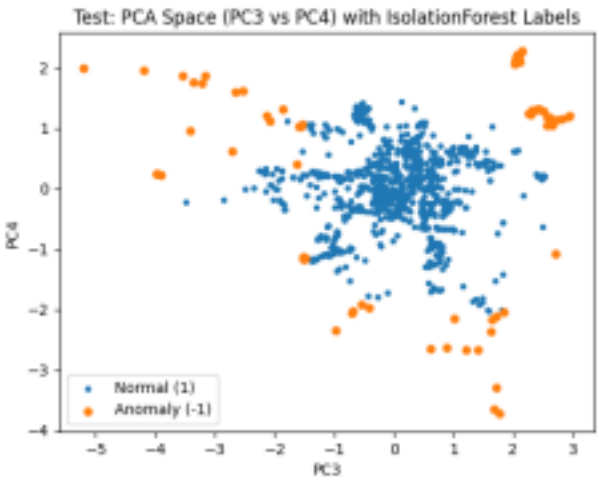
The most severe anomalies in the test dataset were inspected by ranking observations using the Isolation Forest's continuous anomaly score. The code sorts test_out by if_score_samples in ascending order and displays the top 25 rows. Since sklearn's Isolation Forest assigns lower score_samples() values to more anomalous observations, the rows shown represent the most extreme test-set outliers identified by the model. Many of the top-ranked test anomalies exhibit large-magnitude PCA component values, such as very negative PC4 combined with high PC5, or extreme negative values in PC2/PC3, suggesting these points lie far from the dense region of normal behavior in the PCA feature space.

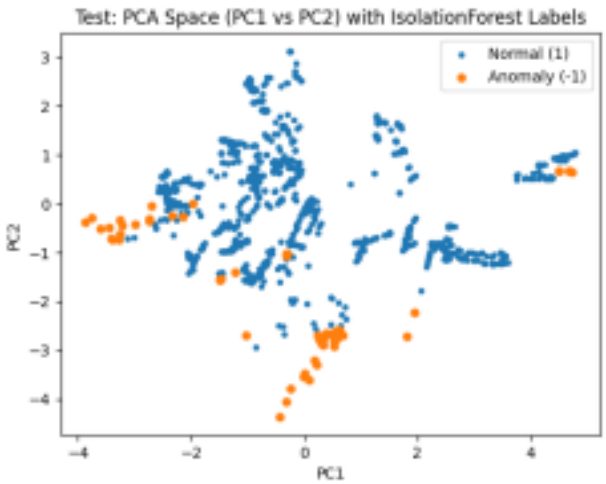
Graphical Analysis of Anomalies (Train):



In the training PCA plots (PC1–PC2 and PC3–PC4), the normal observations form dense central clusters while the Isolation Forest anomalies appear mostly along the edges and in low-density regions of the projected space. This is consistent with Isolation Forest behavior: points that are easier to isolate (typically far from the bulk of the data or on sparse boundaries) are labeled -1.

Graphical Analysis of Anomalies (Test):





In the test PCA plots, the same structure holds—normals remain concentrated in the main clusters and anomalies are primarily located on the periphery and in sparser areas rather than inside the densest regions. The similarity between train and test visual patterns suggests the model's anomaly boundary is stable and generalizes to unseen data rather than flagging points arbitrarily.